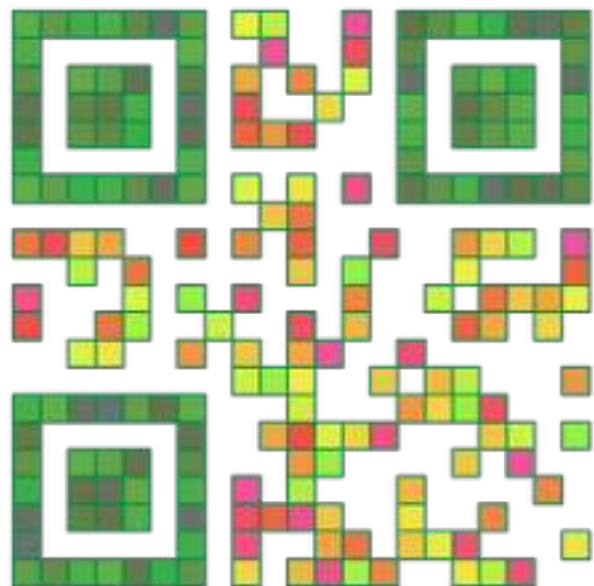


When code reacts to data

CodeMesh 4 Dec 2013



@jessitron





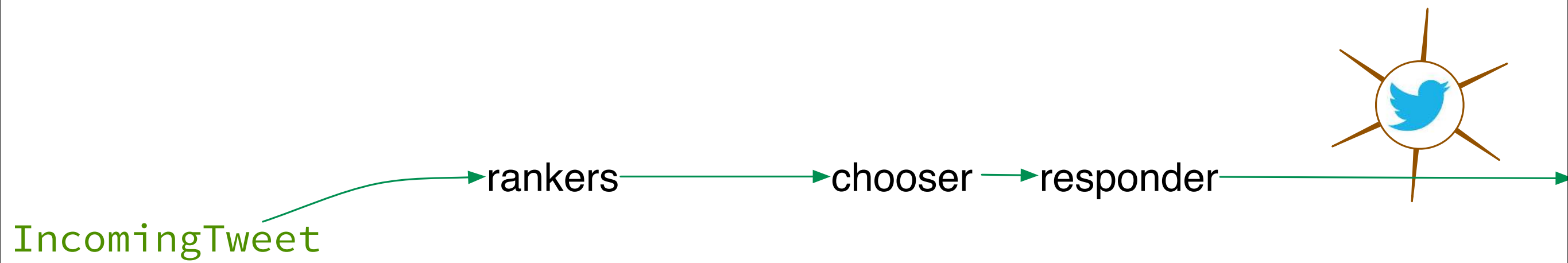


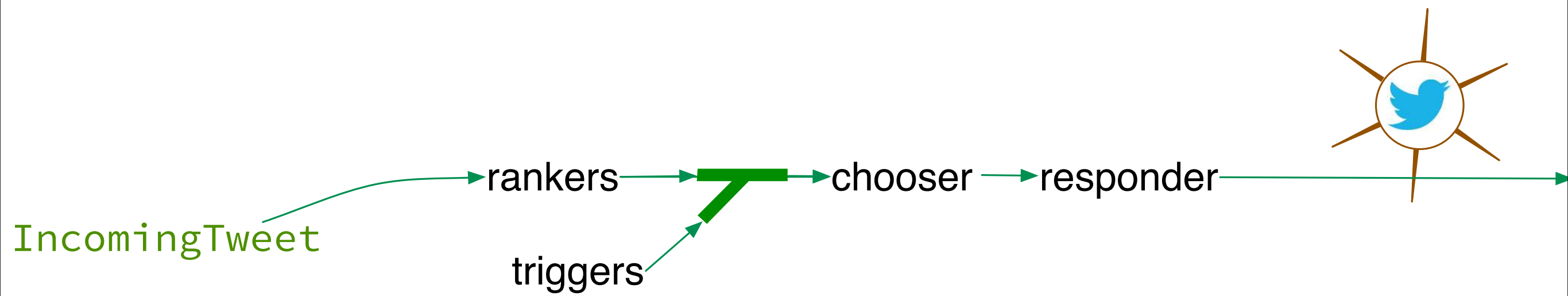
scalaz-stream

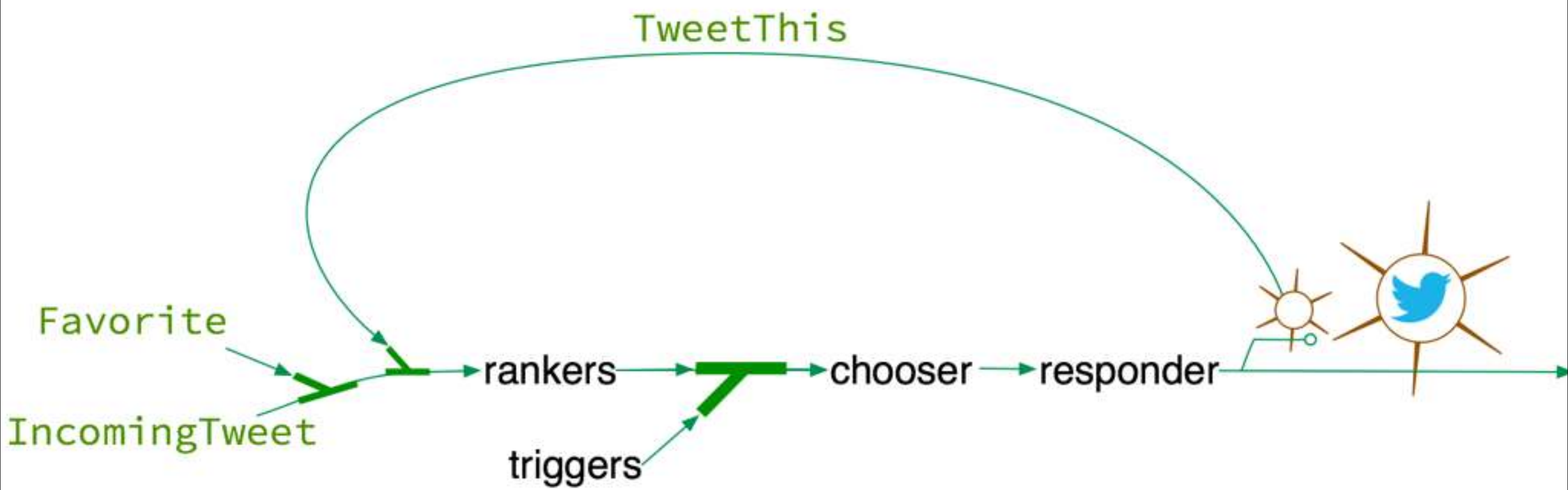
1. compositional
2. expressive
3. resource safe
4. fast

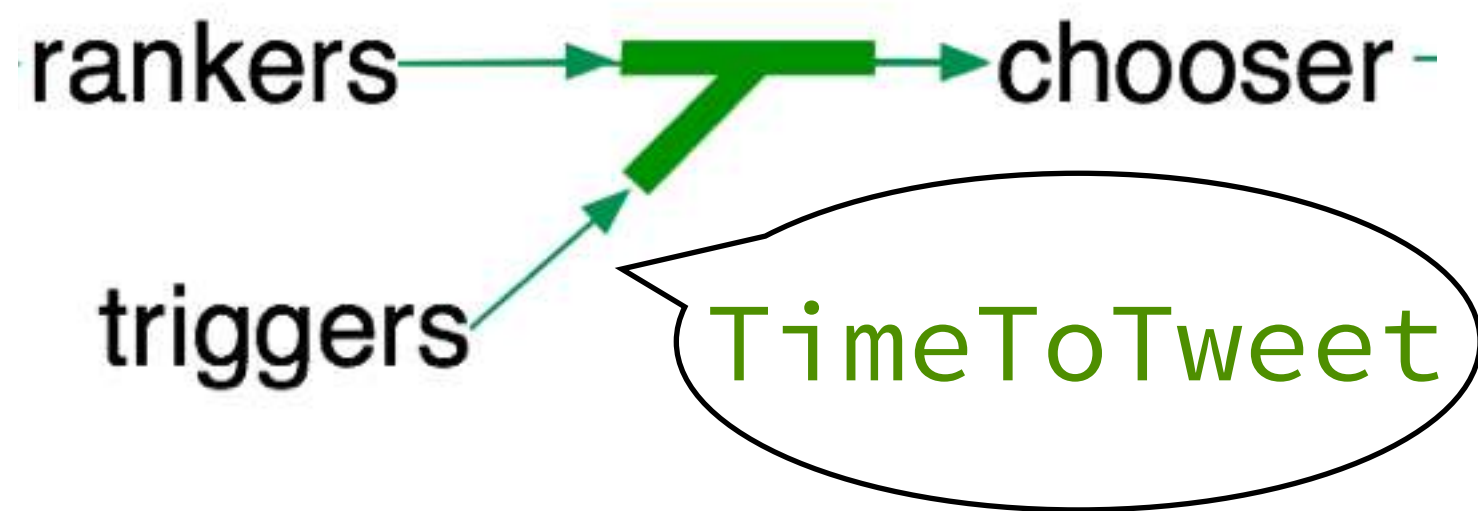


@Con4mationBison









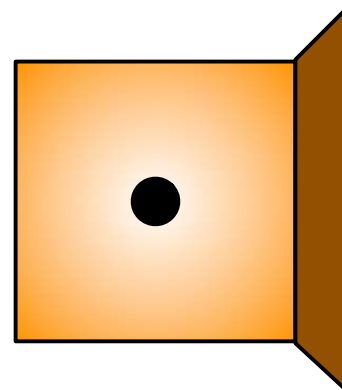
rankers →

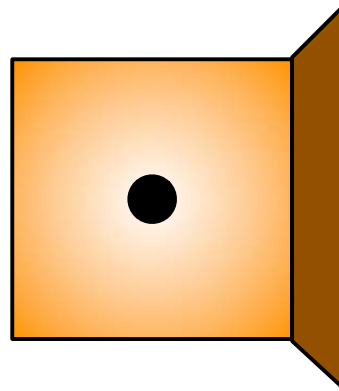


→ chooser

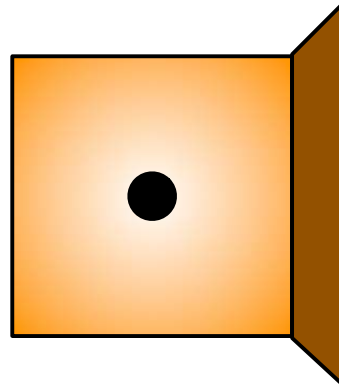
triggers

TimeToTweet

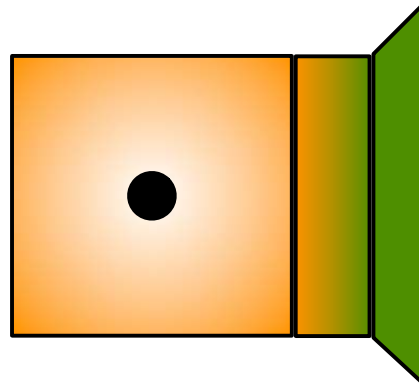




Process [+F[_] , +0]



```
Process.awakeEvery(30 seconds):  
    Process[Task,Duration]
```

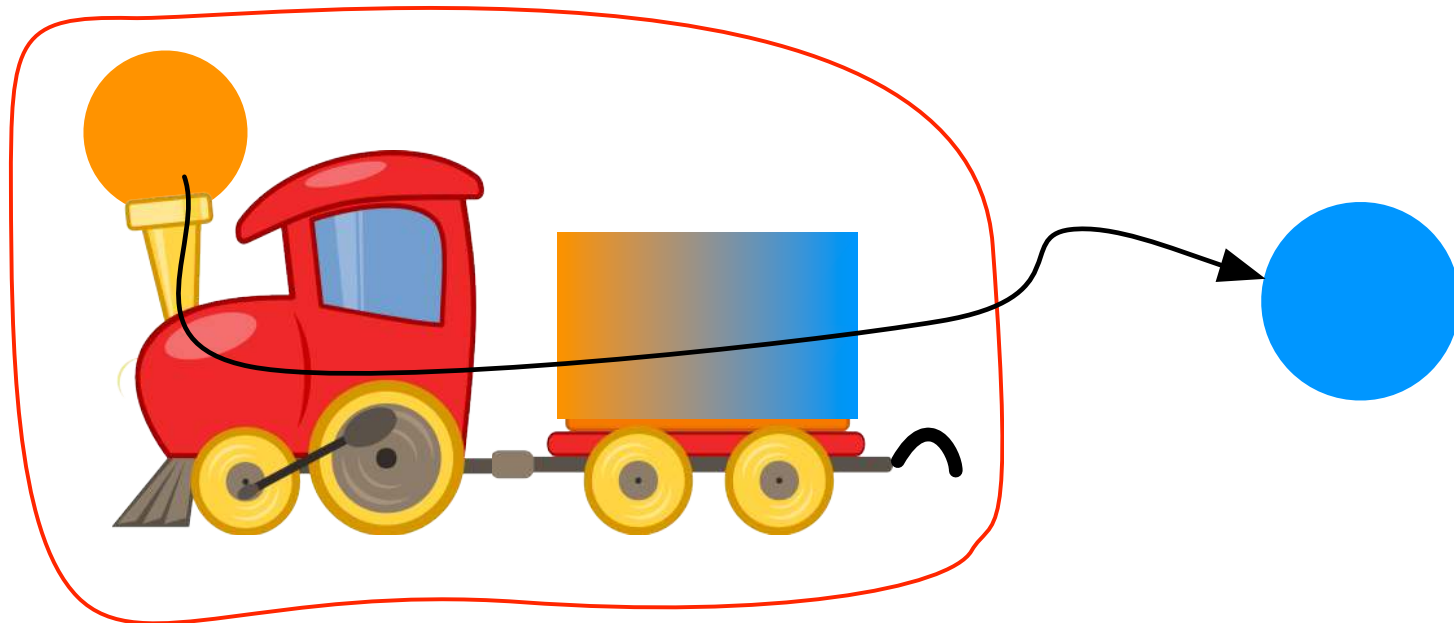


```
Process.awakeEvery(30 seconds) .  
  map { _:Any => TimeToTweet}  
  : Process[Task, TimeToTweet]
```


List[A]



List[B]

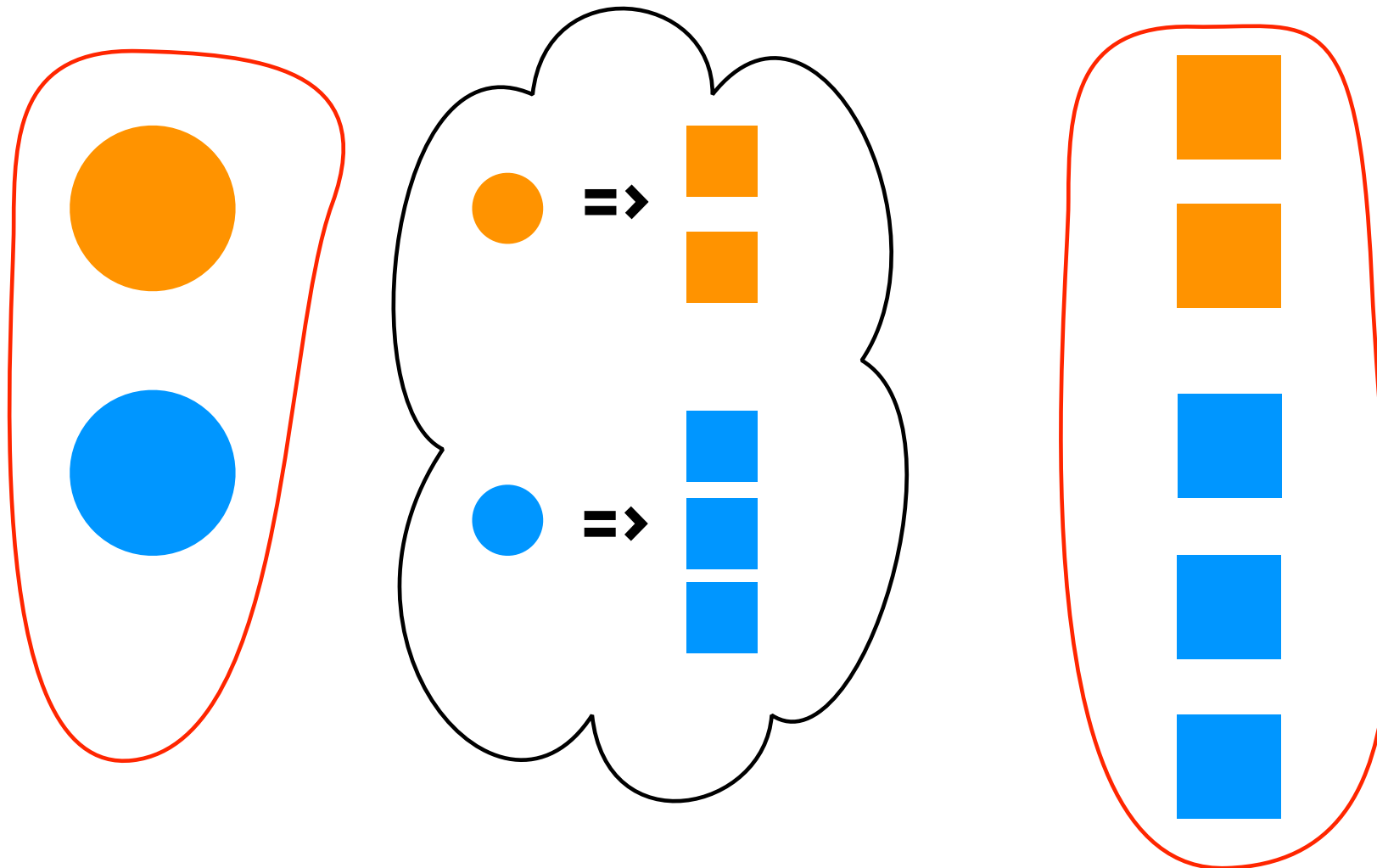


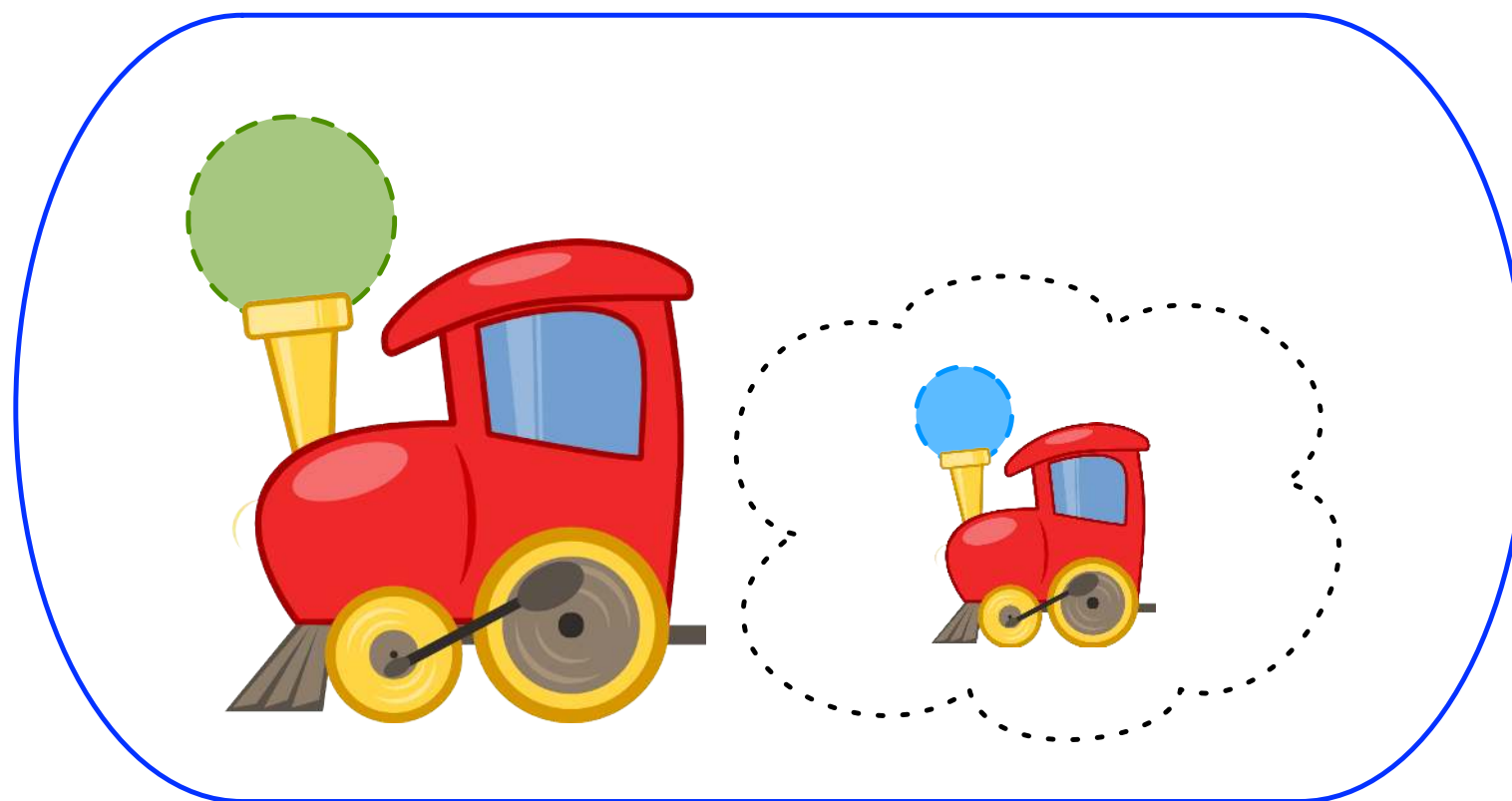
List[A]

flatMap

=>

List[B]



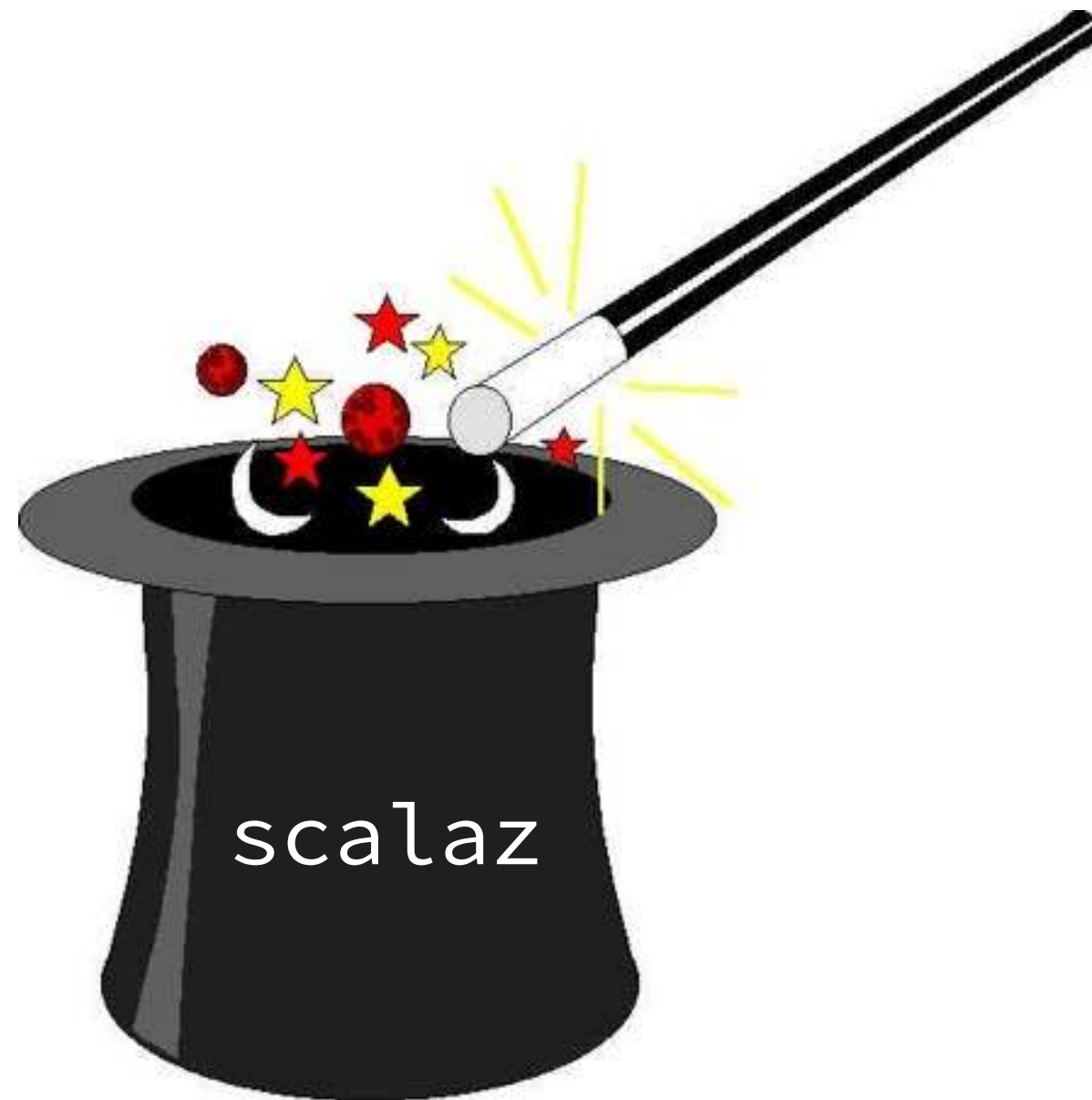


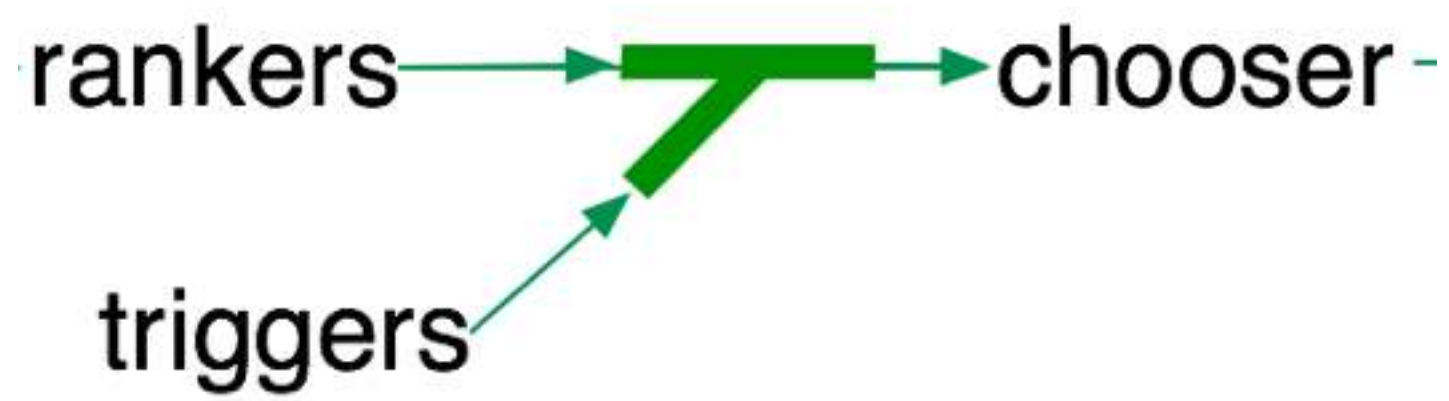
I don't care what
your special power is.
Here's your mission,
now get it done



```
/**
 * Collect the outputs of this Process,
 * given a Monad[F] in which
 * we can catch exceptions...
 */
def runLog(implicit F: Monad[F],
           C: Catchable[F]): F[Seq[O]]
```

```
(implicit F: Monad[F],  
  C: Catchable[F])
```






```
type  
Wye[-I,-I2,+0] = Process[Env[I,I2]#Y,0]
```

type

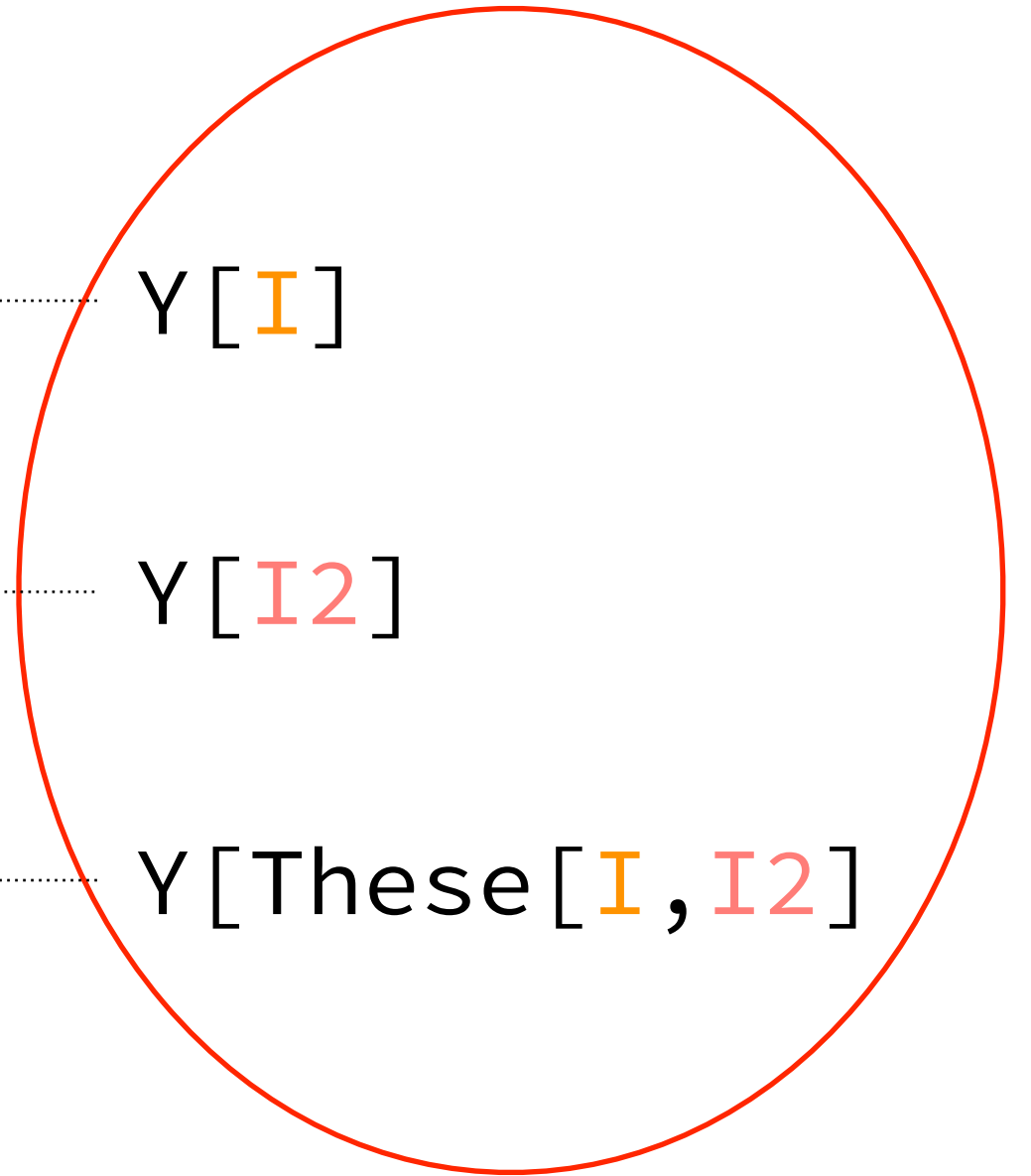
Wye[-I,-I2,+0] = Process[Env[I,I2]#Y,0]

```
case class Env[-I,-I2]() {  
  sealed trait Y[-X]  
  sealed trait T[-X] extends Y[X]  
  sealed trait Is[-X] extends T[X]  
  
  case object Left extends Is[I]  
  case object Right extends T[I2]  
  case object Both extends Y[These[I,I2]]  
}
```

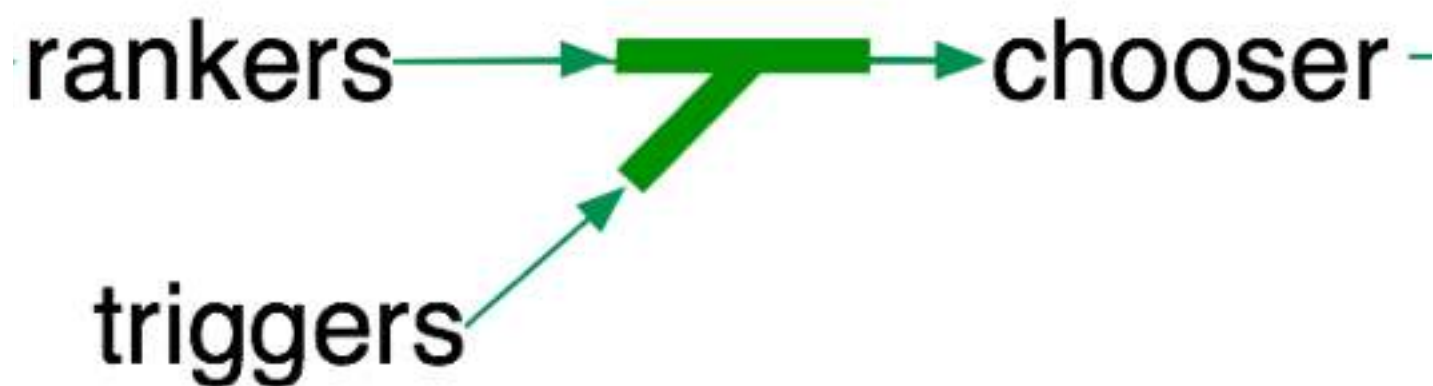
Left Is[I] T[I] Y[I]

Right T[I2] Y[I2]

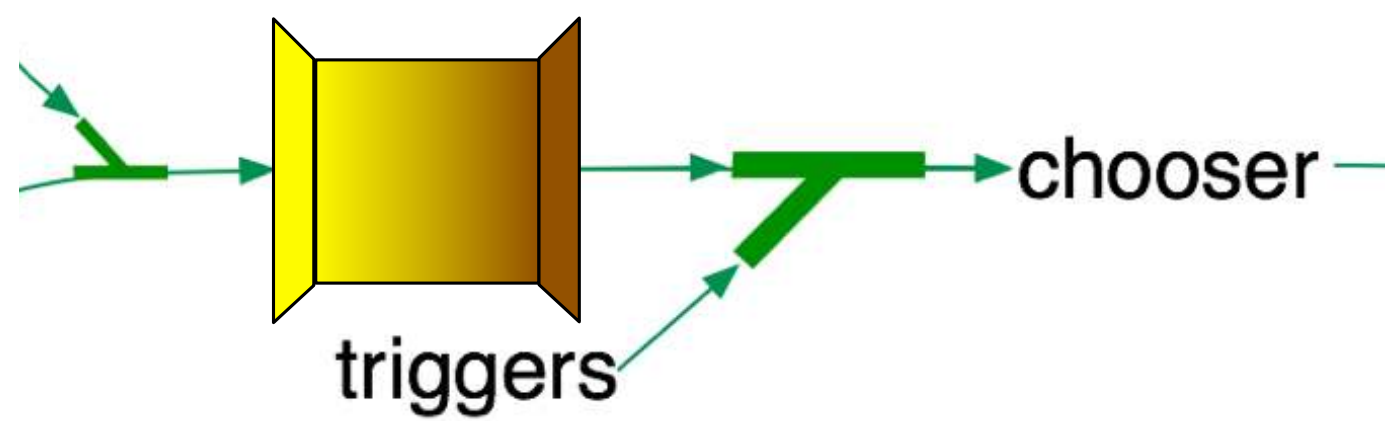
Both Y[These[I, I2]]



Wye[Message, Message, Message]

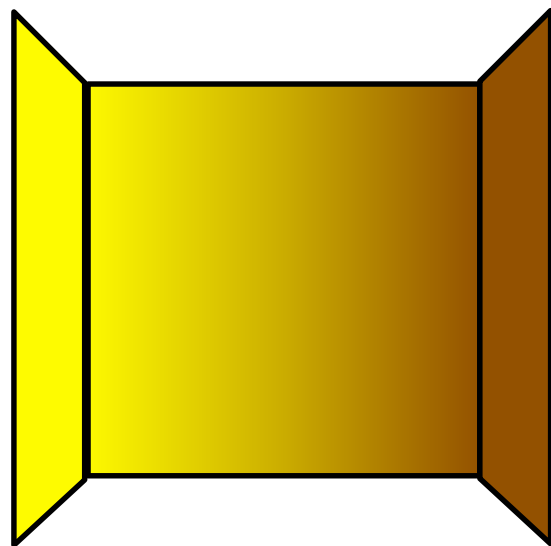


```
ranker.wye(triggers)(wye.merge)
```

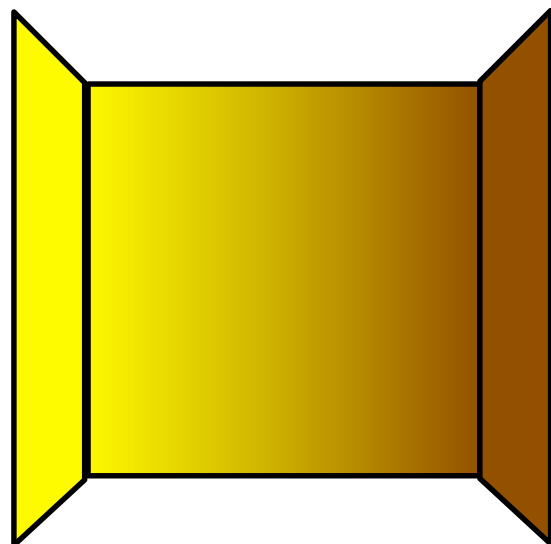



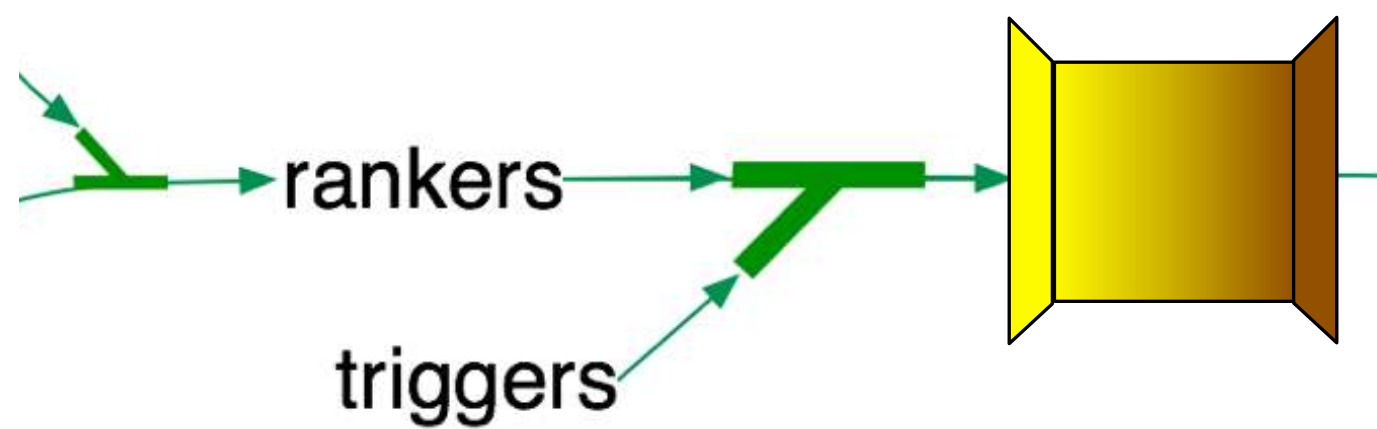
type

`Process1[-I, +0] = Process[Env[I, Any]#Is, 0]`



```
val ranker: Process1[Message, Message] =  
  process1.lift {  
    case i: IncomingTweet =>  
      i.addOpinion(Random.nextDouble, "I agree!")  
    case m => m  
  }
```







give me your
data!

`iterator.next()`



Do this with
your data!

`list.map(a => b)`



Let's talk about
your data.

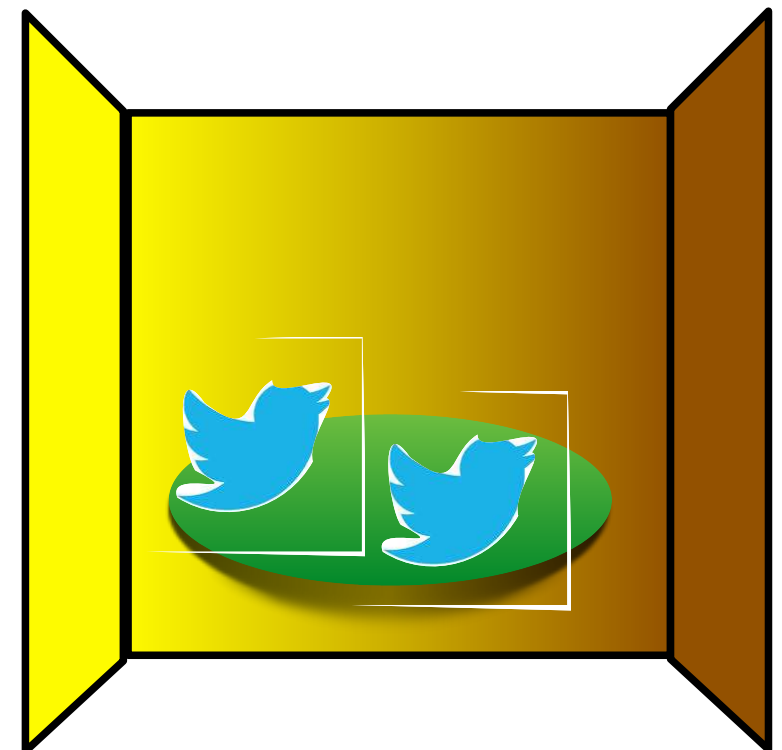
`f: elem => Instruction`

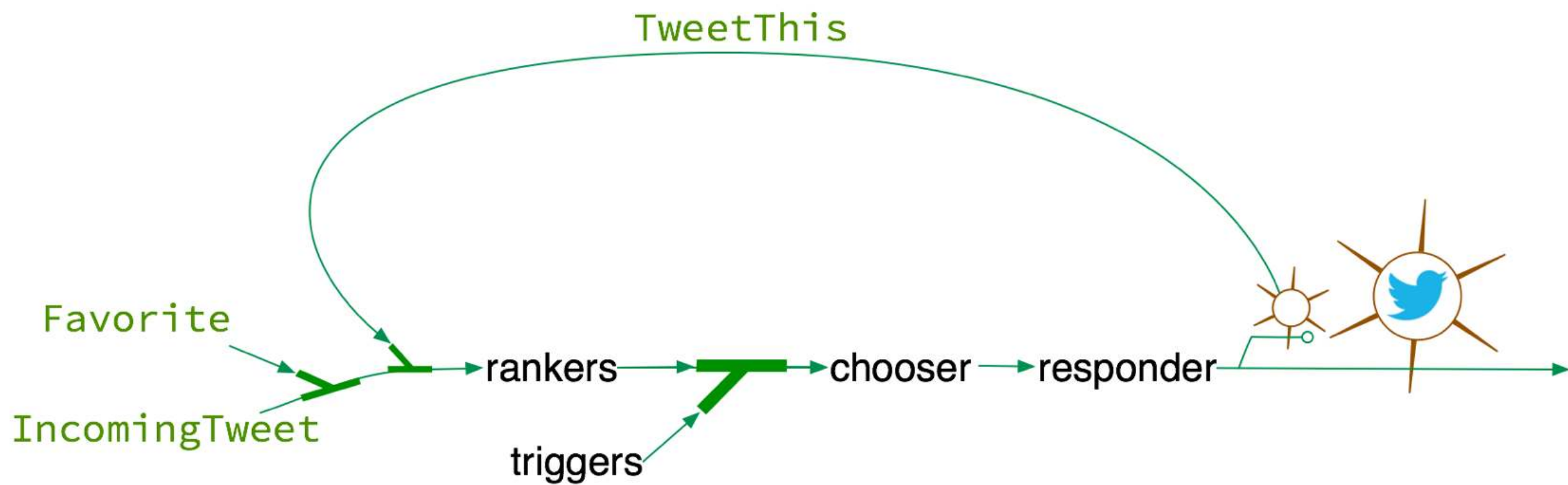
Halt(**cause**: Throwable)

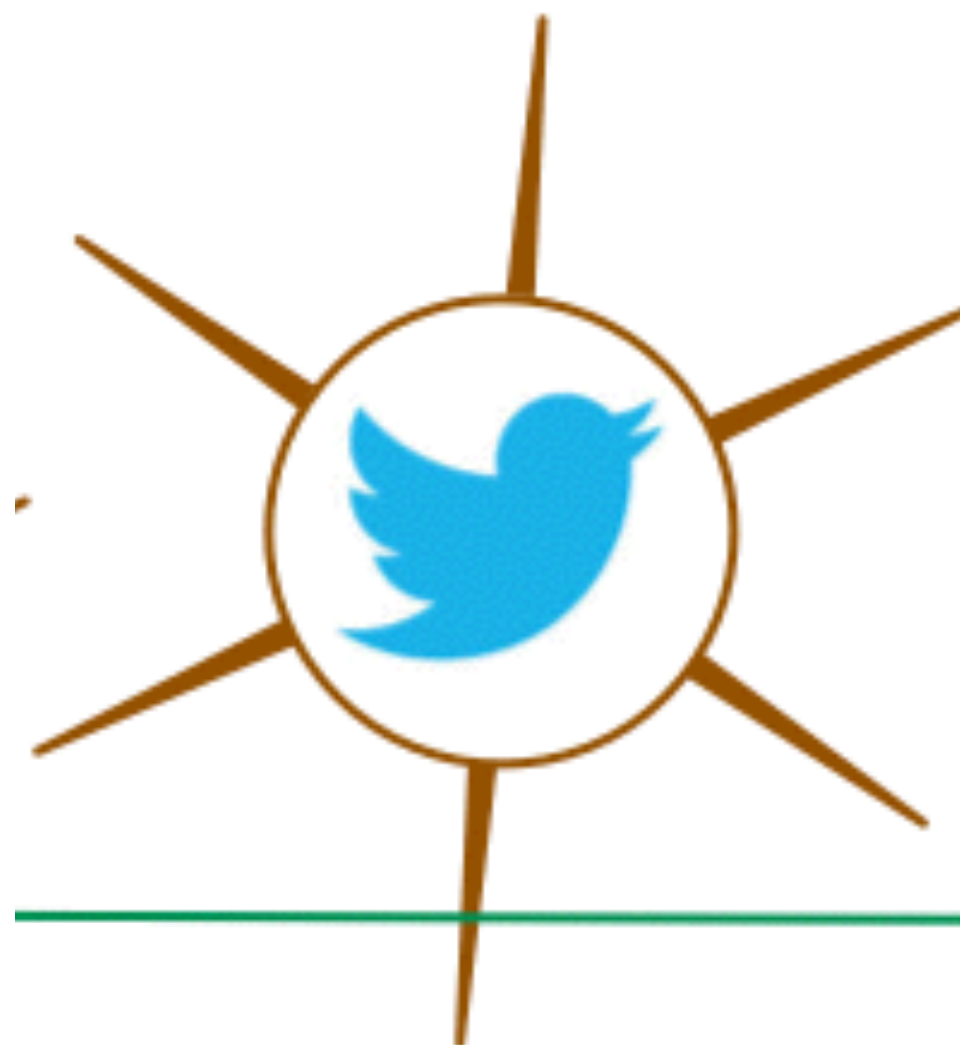
Emit(
 head: Seq[0],
 tail: Process[F,0])

Await[F[_],A,+0] (
 req: F[A],
 recv: A => Process[F,0],
 fallback1: Process[F,0] = halt,
 cleanup1: Process[F,0] = halt)

```
def tweetPicker = {  
  def go(pool: TweetPool): Process1[Message, Message] =  
    await1[Message] flatMap {  
      case i: IncomingTweet => go(pool.absorb(i))  
      case TimeToTweet =>  
        val (newMessage, newPool) = pool.findBest  
        emit(newMessage) ++ go(newPool)  
      case m => emit(m) ++ go(pool)  
    }  
  go(new TweetPool())  
}
```





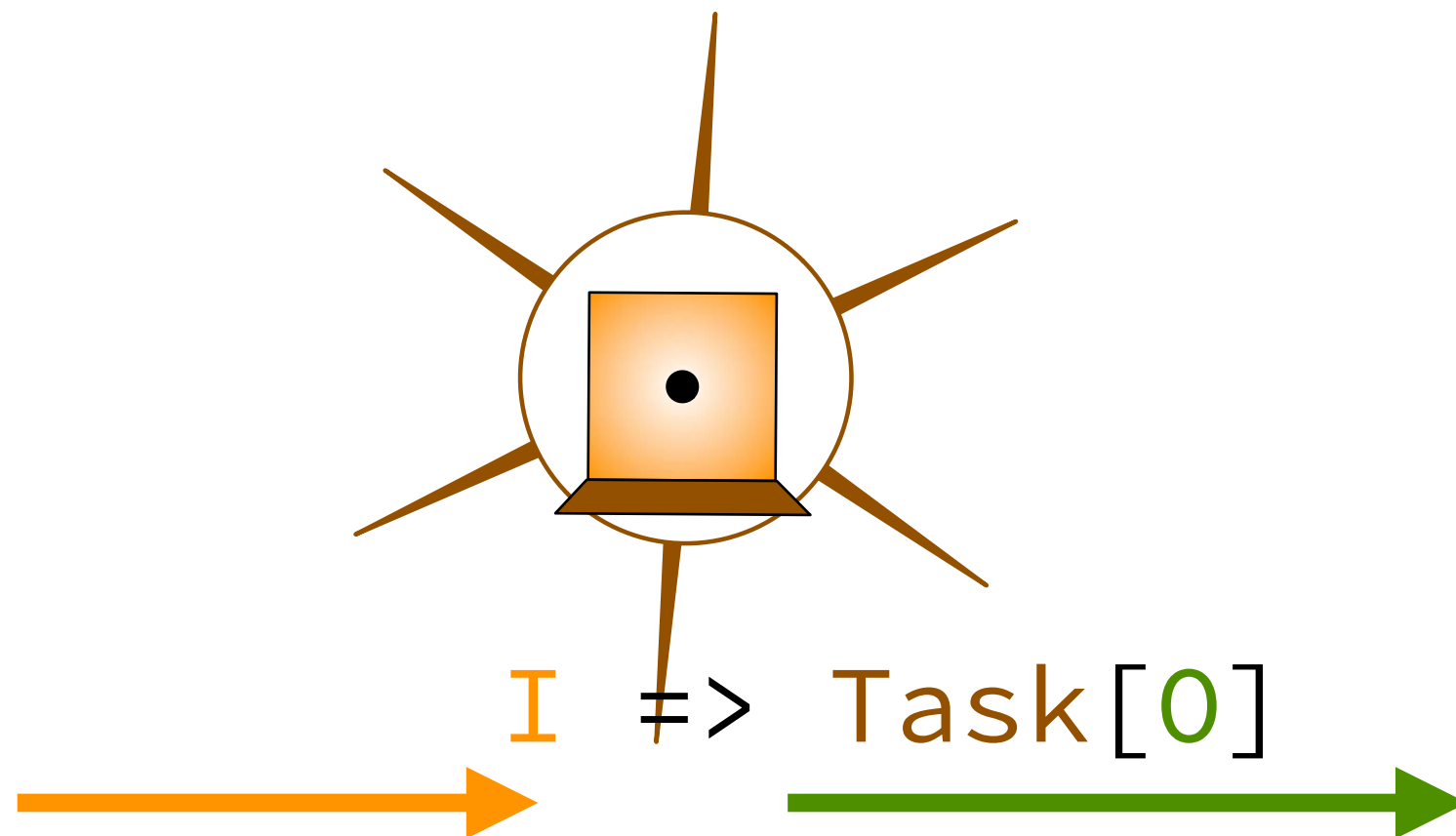


type

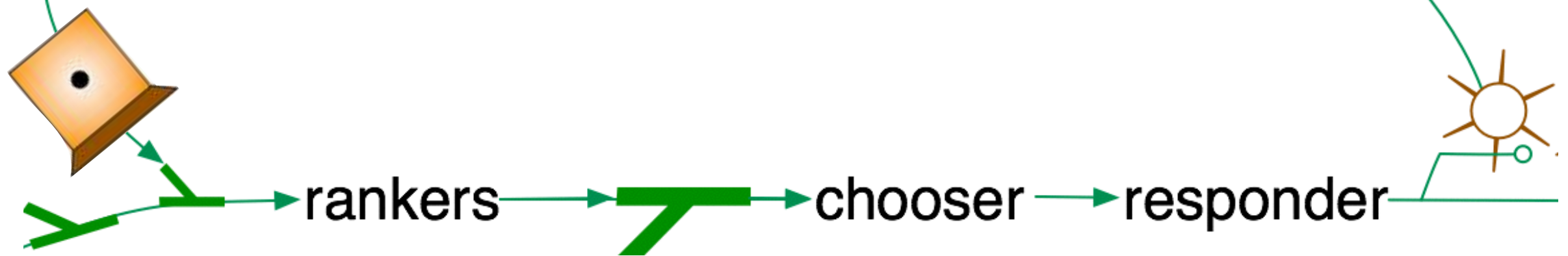
`Channel[+F[_], -I, 0] = Process[F, I=>F[0]]`

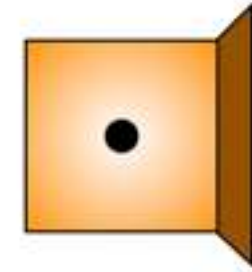
type

$\text{Channel}[+F[_], -I, 0] = \text{Process}[F, I \Rightarrow F[0]]$

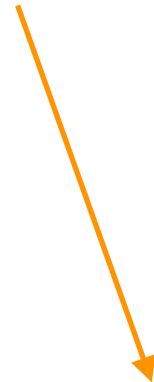


TweetThis



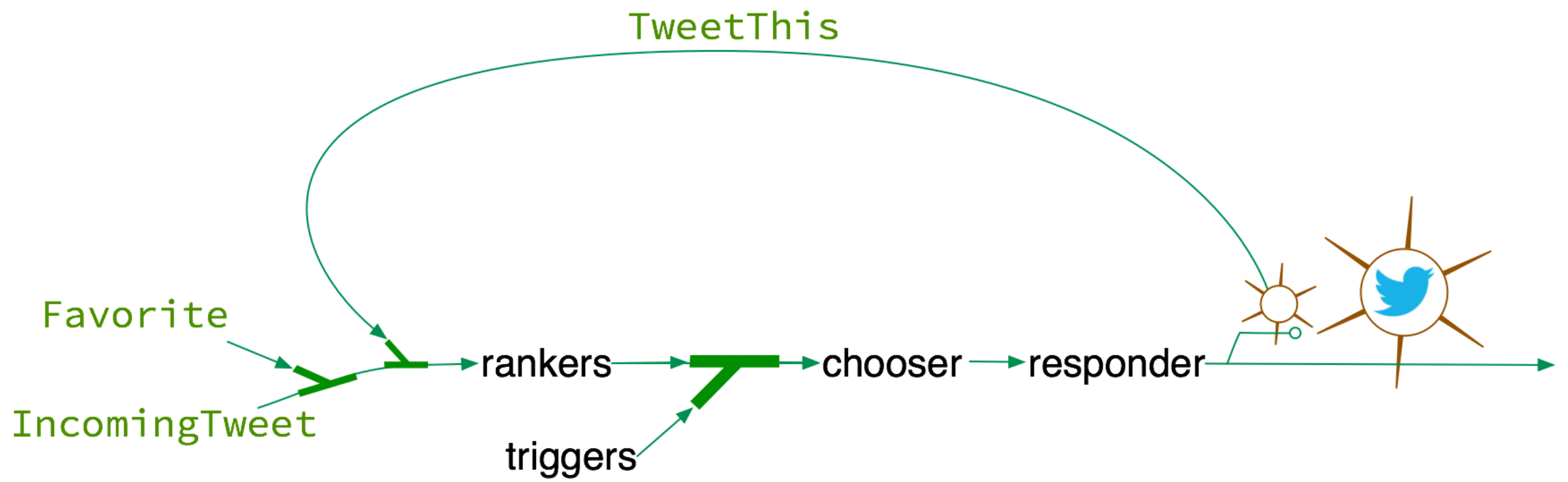


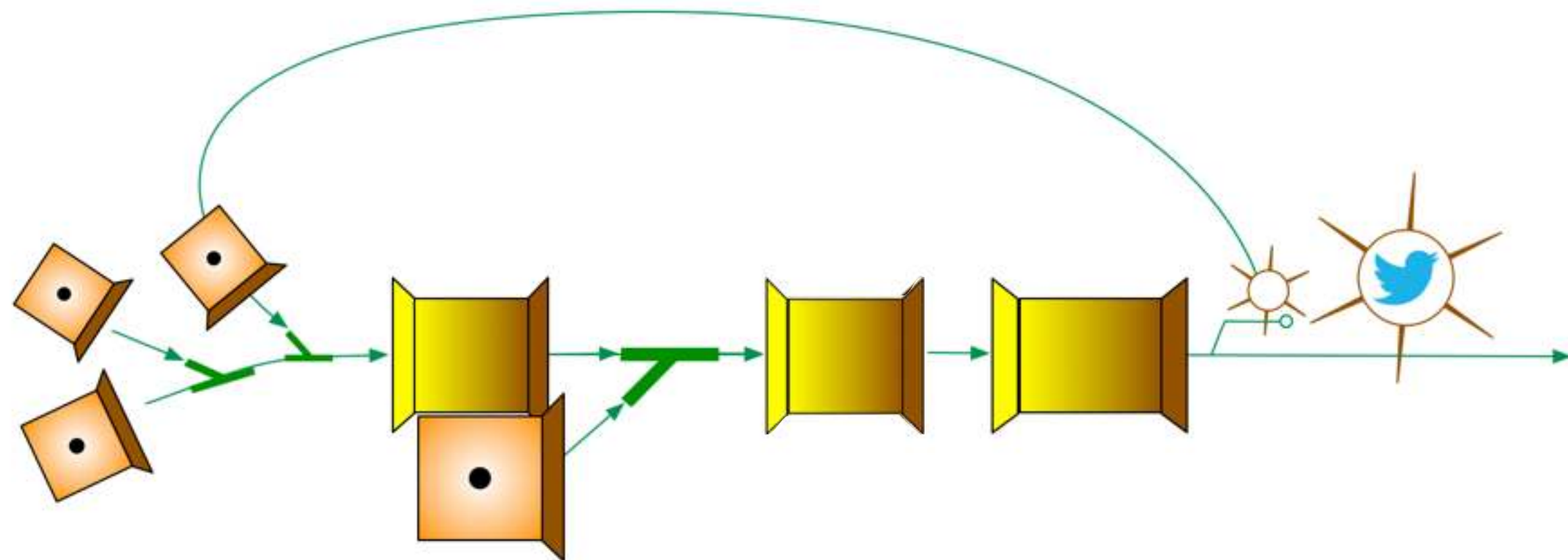
```
async.queue: (Queue[A], Process[Task, A])
```

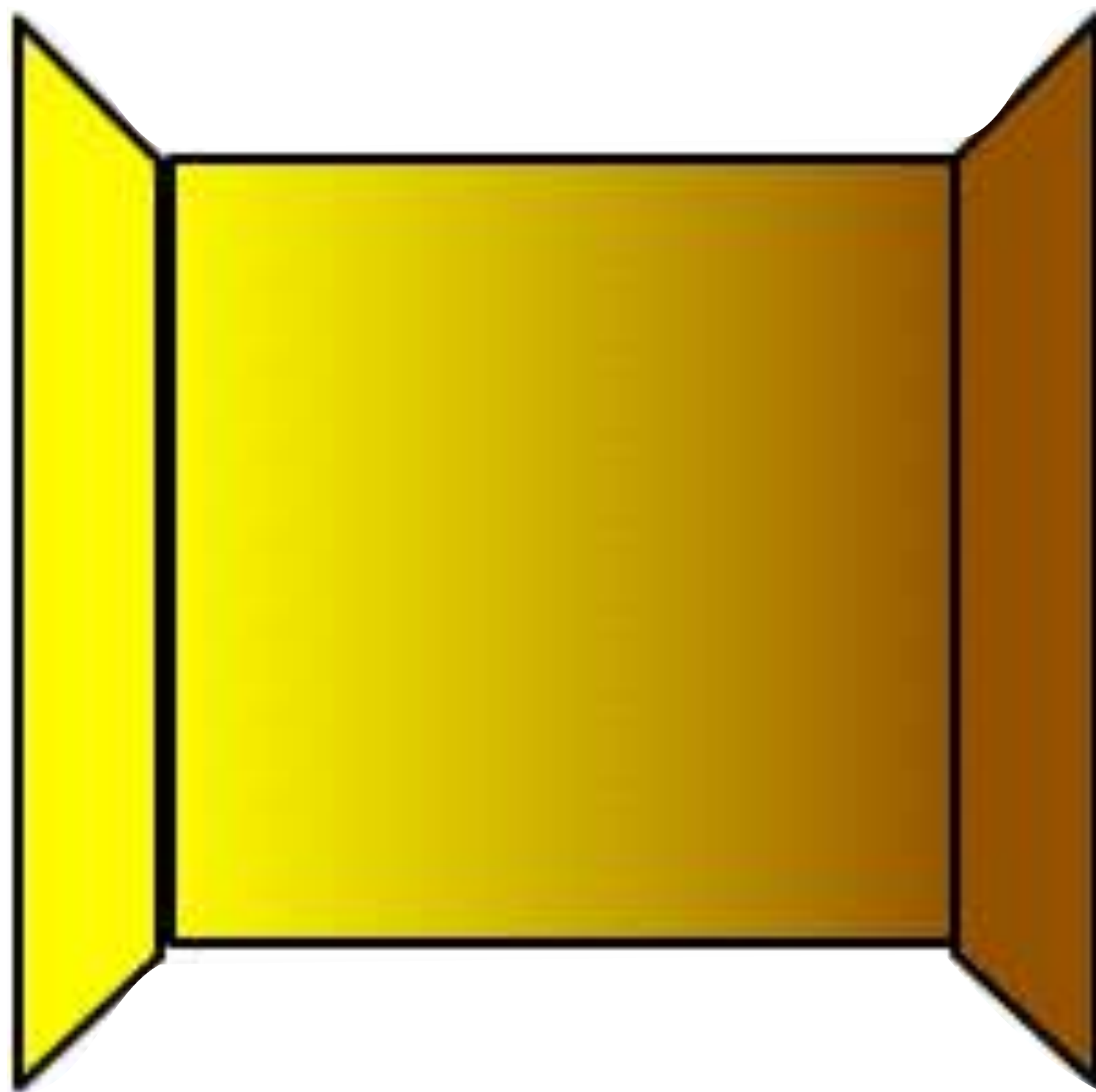


```
async.toSink(Queue[A]): Sink[Task, A]
```













scalaz-stream

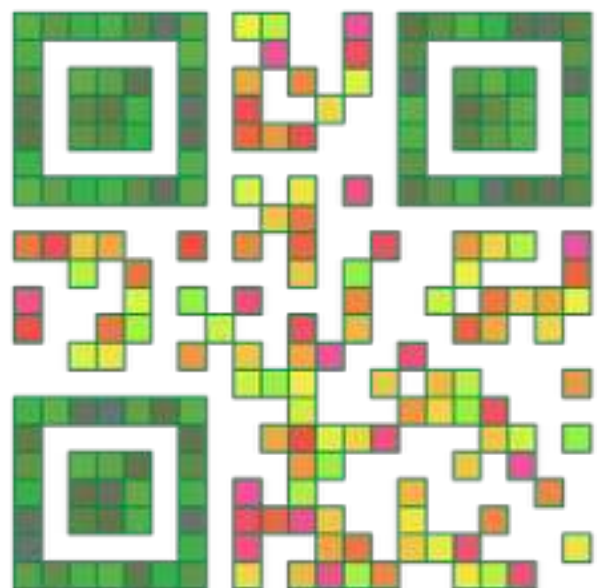
1. compositional
2. expressive
3. resource safe
4. fast

Functional Programming in

MEAP

 MANNING

Paul Chiusano
Rúnar Bjarnason



Jessica Kerr
blog.jessitron.com

@jessitron

github.com/jessitron/bison

github.com/scalaz/scalaz-stream